Compiling Structured Operational Semantics to Executable OCaml Code

Linglong Meng
University of Minnesota, Twin Cities

2025 Midwest Programming Languages Summit
December 13, 2025 — University of Minnesota, Twin Cities

Introduction

When we write down specifications for a programming langauges and reason about it, we usually write down on fancy paper specs, then in someway turn into implementation. However, here we are trying to build a compiler that can directly translate into OCaml code.

Background

The compiler is build on a existed compiler build upon by UMN graudted Ph.D Dawn Michaelson. The Compiler Sterling can generate Prolog (as one of its many other backends like extensibella but that's for reasoning framework).

Spec Language Overview

Constructor declaration:

$$\begin{aligned}\tau ::= &\ \alpha \\ \mid &\ \beta \\ \mid &\ \kappa(\gamma) \\ \mid &\ \delta(\epsilon, \zeta, \eta)\end{aligned}$$

Overall structure:

$$\frac{\begin{array}{c}\langle\text{premise 1}\rangle \\ \vdots \\ \langle\text{premise n}\rangle\end{array}}{\langle\text{conclusion}\rangle}\ [\text{rule name}]$$

Judgement declaration, you can see it as function signature.

Example

Provide a concrete example of your work here.

Results

Present your main results or findings here.

Implementation

Discuss implementation details, such as:

- OCaml code generation strategy
- Key design decisions
- Performance considerations

Evaluation

Include evaluation metrics, benchmarks, or case studies.

Conclusion & Future Work

Summarize your contributions and outline future directions.

References

- Author1. Title1. Conference/Journal, Year.
- Author2. Title2. Conference/Journal, Year.

hello

Contact: meng0181@umn.edu